

Clase 091 — Inyección SQL: fundamentos

Parte: 4 — Seguridad de aplicaciones web · Fuente: *The Web Application Hacker's Handbook (Stuttard & Pinto)* 🕒 Duración estimada: 120 min · Nivel: Intermedio

Objetivo

Comprender a fondo la **inyección SQL (SQLi)**: por qué ocurre, cómo detectarla y cómo explotarla en su forma clásica (in-band). Es la vulnerabilidad emblema de la categoría A03 Injection y una de las de mayor impacto: puede exponer bases de datos completas.

⚠️ **Ética:** todo lo aquí descrito se practica únicamente en laboratorios propios (DVWA, Juice Shop) o con autorización explícita por escrito. Inyectar SQL en sistemas ajenos es un delito.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Explicar** cómo la concatenación de entradas produce SQLi.
- 2. **Detectar** puntos inyectables con pruebas de error y booleanas.
- 3. **Explotar** SQLi con `UNION SELECT` para extraer datos.
- 4. **Enumerar** esquema, tablas y columnas de la base de datos.
- 5. **Recomendar** la corrección: consultas parametrizadas.

Temas

#	Tema	Por qué importa
1	Cómo se construye una query vulnerable	Es la causa raíz
2	Detección: comillas, errores, lógica	Confirmar el punto inyectable
3	UNION-based injection	Extracción directa de datos
4	Enumeración del esquema	Saber qué robar y de dónde
5	Bypass de autenticación con SQLi	Impacto inmediato
6	Comentarios y sintaxis por motor	MySQL, MSSQL, Postgres difieren
7	Remediación: prepared statements	Cierre correcto del fallo

Definiciones y características

- **Inyección SQL:** inserción de sintaxis SQL en una entrada que se concatena a una consulta. Característica: rompe la separación entre datos y código.

- **In-band SQLi**: los resultados vuelven por el mismo canal (la respuesta). Característica: la variante más fácil de explotar.
- **UNION-based**: usa `UNION SELECT` para añadir filas controladas. Característica: requiere igualar número y tipo de columnas.
- **Error-based**: fuerza mensajes de error que filtran datos. Característica: depende de que la app muestre errores.
- **Prepared statement**: consulta con parámetros ligados. Característica: separa código de datos y elimina la SQLi.
- **Comentario SQL** (`--` , `#` , `/* */`): trunca el resto de la query. Característica: clave para bypass de login.

Herramientas y preparación

- **DVWA** (nivel Low y Medium) o Juice Shop.
- **Burp Suite** para interceptar y editar parámetros.
- Cliente SQL para inspeccionar la base de datos y comprobar tu progreso.

```
# DVWA con Docker
docker run --rm -d -p 80:80 vulnerables/web-dvwa
# Login por defecto admin/password, nivel de seguridad "Low"
```

Laboratorio guiado

 Solo en DVWA/Juice Shop propios.

1. En DVWA → *SQL Injection*, introduce `1` y observa la consulta normal.
2. Prueba `1'` y busca un error SQL: confirma que el input llega crudo a la query.
3. Determina el número de columnas con `1' ORDER BY 1-- -` , `ORDER BY 2-- -` , hasta que falle.
4. Extrae datos con UNION:

```
1' UNION SELECT user, password FROM users-- -
```

5. Enumera el esquema con `information_schema`:

```
1' UNION SELECT table_name, NULL FROM information_schema.tables-- -
```

6. Prueba un **bypass de login** en el formulario de autenticación: `admin'-- -` .
7. Documenta cada payload, la respuesta y el dato exfiltrado.
8. Repite en nivel "Medium" (input por POST, comillas escapadas) y observa las diferencias.

Ejercicios

1. Determina el motor de base de datos por su sintaxis de comentarios y errores.
2. Extrae el hash de la contraseña de `admin` y crackéalo (offline, con hashcat) en tu lab.
3. Consigue el nombre de la base de datos actual con `database()` .
4. Explica por qué `ORDER BY` ayuda a contar columnas.

5. Escribe la versión parametrizada (segura) de la query vulnerable en PHP y en Python.
6. Diferencia UNION-based de error-based con un ejemplo propio.

Reto verificable

Extrae **todos los usuarios y hashes** de la tabla `users` de DVWA vía UNION SQLi y luego reescribe la consulta backend de forma segura. **Criterio de aceptación:** entregas el listado exfiltrado (evidencia), el payload UNION usado y el código corregido con prepared statements que impide la inyección.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
"The used SELECT statements have a different number of columns"	Cuenta mal de columnas; ajusta el UNION
No hay error visible	Errores desactivados; pasa a blind (próxima clase)
Comilla escapada	Nivel Medium filtra <code>'</code> ; prueba numérico o encoding
UNION devuelve tipos incompatibles	Usa NULL en columnas para igualar tipos
Bypass de login no funciona	Sintaxis de comentario incorrecta para el motor

Preguntas frecuentes

 **¿Los ORM me protegen?** En gran medida, si usas sus métodos parametrizados. Pero el SQL crudo dentro de un ORM vuelve a ser vulnerable.

 **¿Escapar comillas es suficiente?** No de forma fiable. La defensa correcta son las consultas parametrizadas; el escaping manual es propenso a errores.

 **¿Por qué `information_schema` es tan útil?** Es el catálogo estándar que describe tablas y columnas; permite mapear la base de datos sin conocerla de antemano.

Referencias

- Stuttard & Pinto, *The Web Application Hacker's Handbook*, cap. 9 (Injecting into the DB).
- OWASP SQL Injection: https://owasp.org/www-community/attacks/SQL_Injection
- PortSwigger SQL injection: <https://portswigger.net/web-security/sql-injection>
- OWASP SQLi Prevention Cheat Sheet.

Siguiente clase

Clase 092 - Inyeccion SQL avanzada y ciega (blind)